UNIVERSIDAD NACIONAL DE LOJA

Facultad de la Energía, las Industrias y los Recursos Naturales no Renovables

GRUPO 5

Juego de luces interactivo con ESP32

Práctica Nro. 002

INTEGRANTES:

Martina Alejandra Maldonado Machuca
Victor Fernando Roa Carrión
Emilio José Galvez Merchan
Wilson Patricio Palma Samaniego

Ciclo: Quinto
Asignatura: Sistemas Digitales

Docente: Ing. Ramiro Rivera
LOJA-ECUADOR

Reporte Técnico de Actividades Práctico-Experimentales Nro. 003

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Martina Alejandra Maldonado Machuca Wilson Patricio Palma Samaniego Victor Fernando Roa Carrión Emilio José Galvez Merchan
Asignatura	Sistemas Digitales
Ciclo	5to Ciclo
Unidad	1 – Microcontroladores para sistemas embebidos
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	2
Título de la Práctica	Juego de luces interactivo con ESP32
Nombre del Docente	Luis Darío Sinche C.
Fecha	23 de octubre de 2026
Horario	Viernes 11h30 a 13h30
Lugar	Laboratorio de computación (simulación en Wokwi y prueba en la placa ESP32 del grupo)
Tiempo planificado en el Sílabo	3 horas (APE, semana 2): 2 h presenciales y 1 h para terminar el informe

2. Objetivo(s) de la Práctica

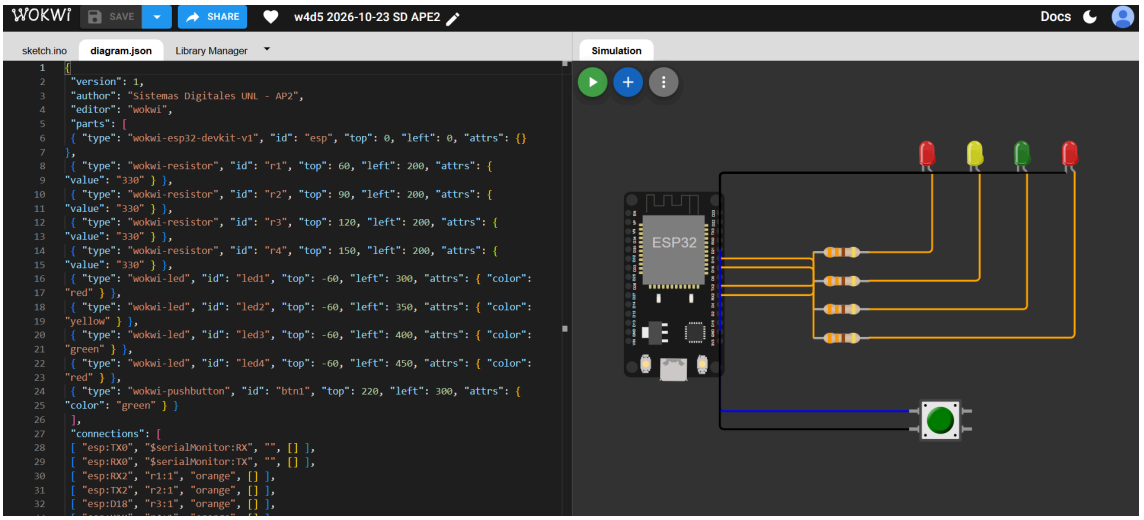
- Programar en el ESP32 un juego de luces con cuatro LEDs cuya velocidad cambia con un pulsador, usando temporización no bloqueante con millis() y la lectura del pulsador con pull-up interna, antirrebote y detección de flanco.

3. Materiales y software usados.

- Pizarra y marcadores
- Proyector multimedia
- Hojas de trabajo impresas para cada fase
- Guía de laboratorio con ejercicios prácticos
- Enlaces de referencia:
- Enlaces de referencia: https://acortar.link/lC817u (El amanecer de la RED), https://acortar.link/0lcDSp (Práctica Estándares), https://acortar.link/73rKsj (Práctica Modelos TCP y OSI)
- Video: https://www.youtube.com/watch?v=PjegxDO19TA

4. Esquema de conexiones

4.1. Captura de Woki



4.2. Tabla de conexiones

Componente	GPIO	Serigrafía en la placa	Conexión
LED 1	16	RX2	GPIO → resistencia 330 Ω → ánodo (pata larga); cátodo (pata corta) → GND
LED 2	17	TX2	igual que el LED 1
LED 3	18	D18	igual que el LED 1
LED 4	19	D19	igual que el LED 1
Pulsador	21	D21	una pata al GPIO21 y la pata del lado opuesto a GND; sin resistencia externa (usa la pull-up interna)
Tierra	—	GND	línea azul (−) de la protoboard; ahí van los cátodos y el pulsador

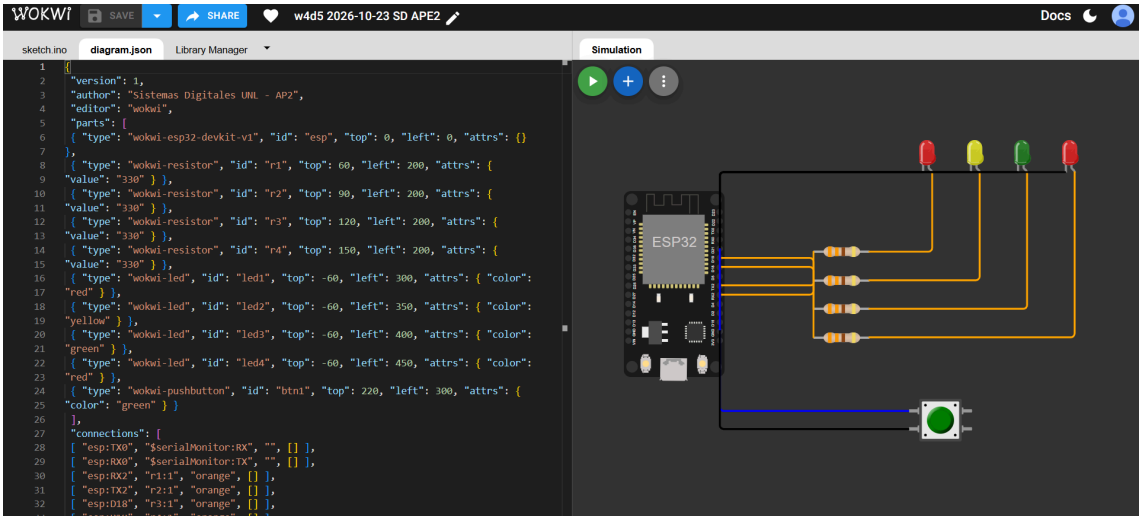
5. Resultados

5.1 Parte A. Circuito y secuencia de luces (Wokwi)

A1. Entren a wokwi.com y creen un proyecto nuevo de ESP32.
https://wokwi.com/projects/478692759417332737

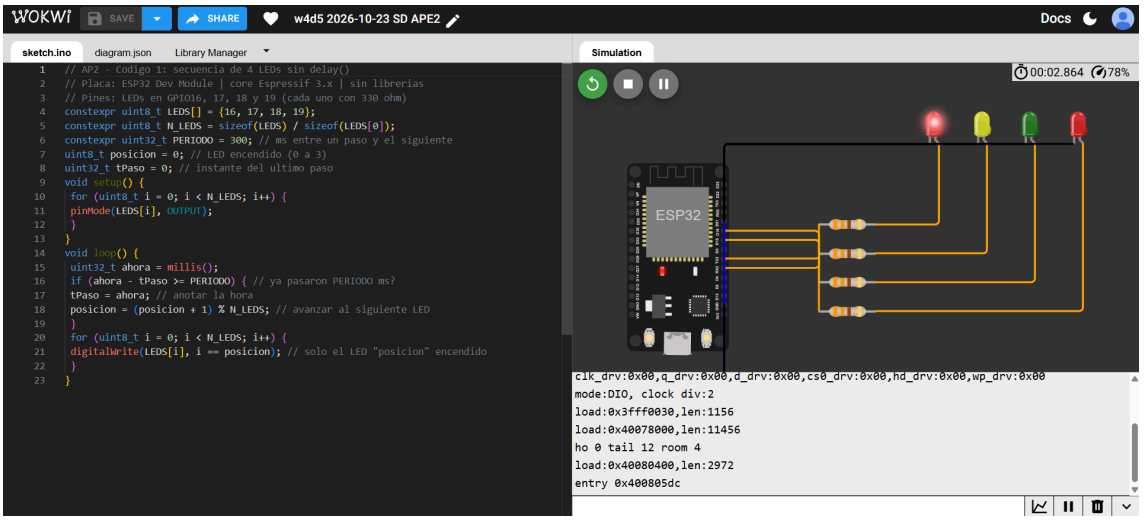
A2. Abran la pestaña diagram.json, borren todo su contenido y peguen el Anexo A.

Aparecen la placa, los cuatro LEDs, las resistencias y el pulsador ya cableados.



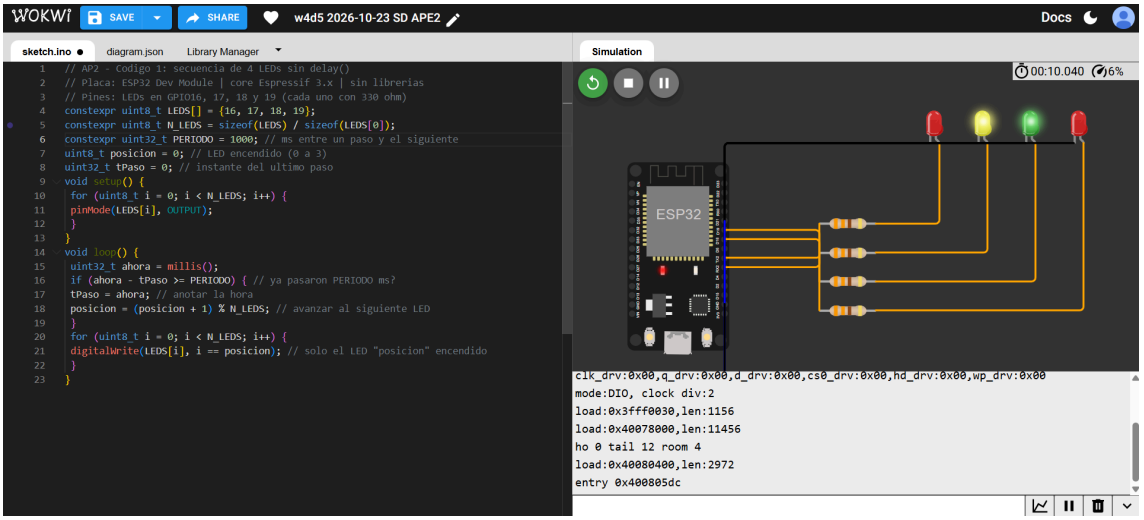
A3. Revisen que cada cable coincida con la Tabla de conexiones. Ubiquen en la placa los pines RX2, TX2, D18, D19 y D21.

A4. Abran la pestaña sketch.ino, borren su contenido, peguen el Código 1 y presionen ▶ (Start). Resultado esperado: un solo LED encendido a la vez, que avanza 16 → 17 → 18 → 19 → 16… cada 0,3 s.

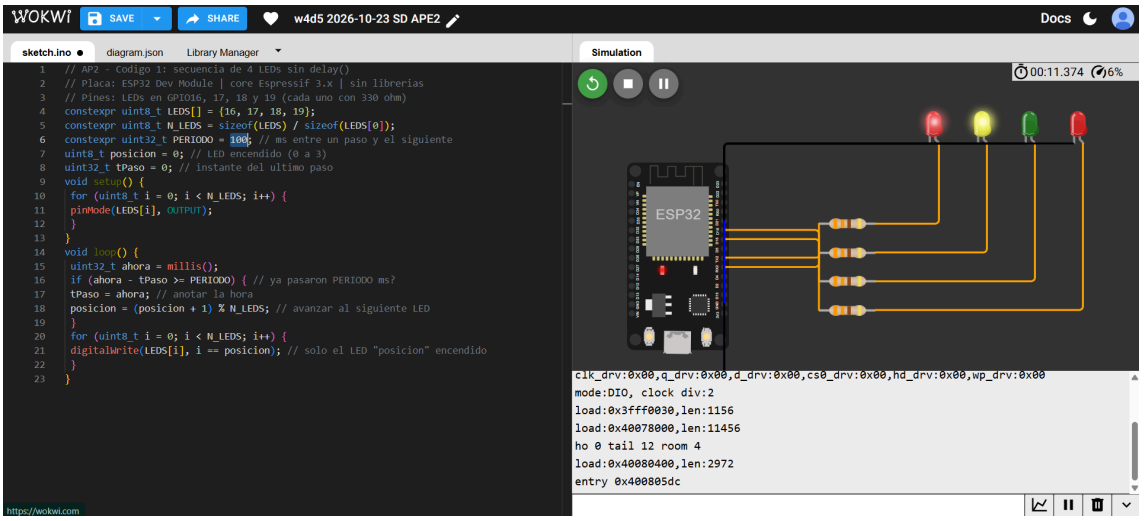


A5. Cambien PERIODO a 1000 y luego a 100. Ejecuten cada vez y completen la Tabla A.

Periodo de 1000:



Periodo de 100:



A6. Vuelvan a dejar PERIODO = 300 y tomen una captura de pantalla del circuito funcionando.

Periodo de 300:

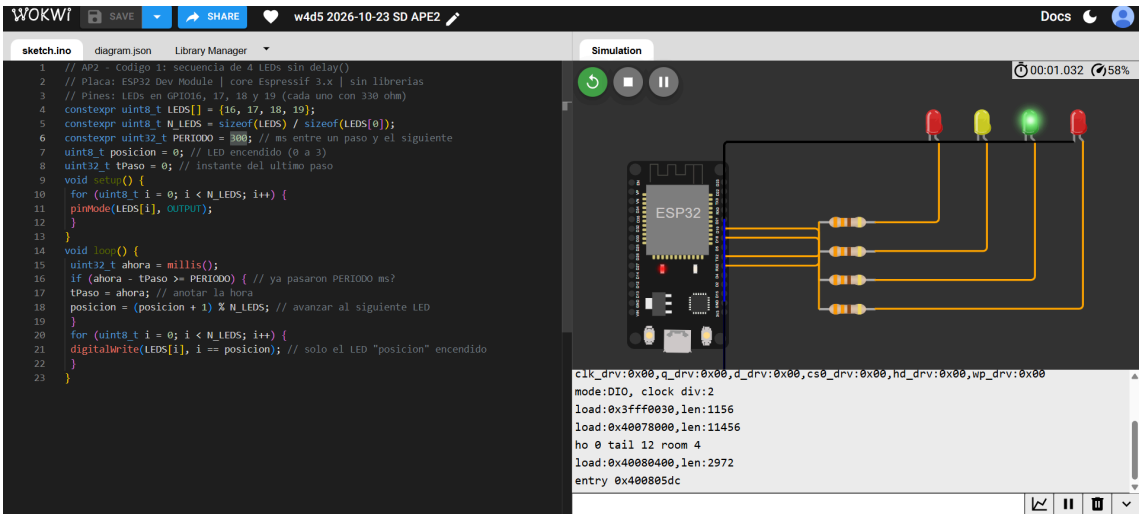


Tabla A. Efecto del periodo

Periodo (ms)	¿Qué observan?
1000	Se observa que el tiempo que pasa entre que se enciende un led y el otro es de 1 segundo, es decir (1000 milisegundos).
300	Se observa que el tiempo que pasa entre que se enciende un led y el otro es más rápido respecto al periodo de 1000, esto se debe a que entre un led y el siguiente solo se demora 300 milisegundos.
100	Se observa que el tiempo que pasa entre que se enciende un led y el otro es aún más rápido respecto al periodo de 300, esto se debe a que entre un led y el siguiente solo se demora 100 milisegundos.

5.3 Parte B. El pulsador cambia la velocidad (Wokwi)

B1. En el mismo proyecto, reemplacen el contenido de sketch.ino por el Código 2 y presionen ▶.

https://wokwi.com/projects/478694011532696577

B2. Hagan clic sobre el pulsador varias veces y miren el Monitor Serie. Resultado esperado: cada clic cambia la velocidad y el Monitor Serie imprime Periodo: ... ms una sola vez por clic.

B3. Mantengan presionado el pulsador durante 3 s (clic sostenido). Resultado esperado: la velocidad cambia una sola vez, aunque el botón siga presionado.

B4. Cambien T_REBOTE a 0, ejecuten, hagan 10 clics y cuenten cuántas líneas imprime el Monitor Serie. Anoten el resultado en la Tabla B.

B5. Vuelvan a dejar T_REBOTE = 30, repitan los 10 clics y completen la Tabla

B6. Tomen una captura del Monitor Serie con T_REBOTE = 30.

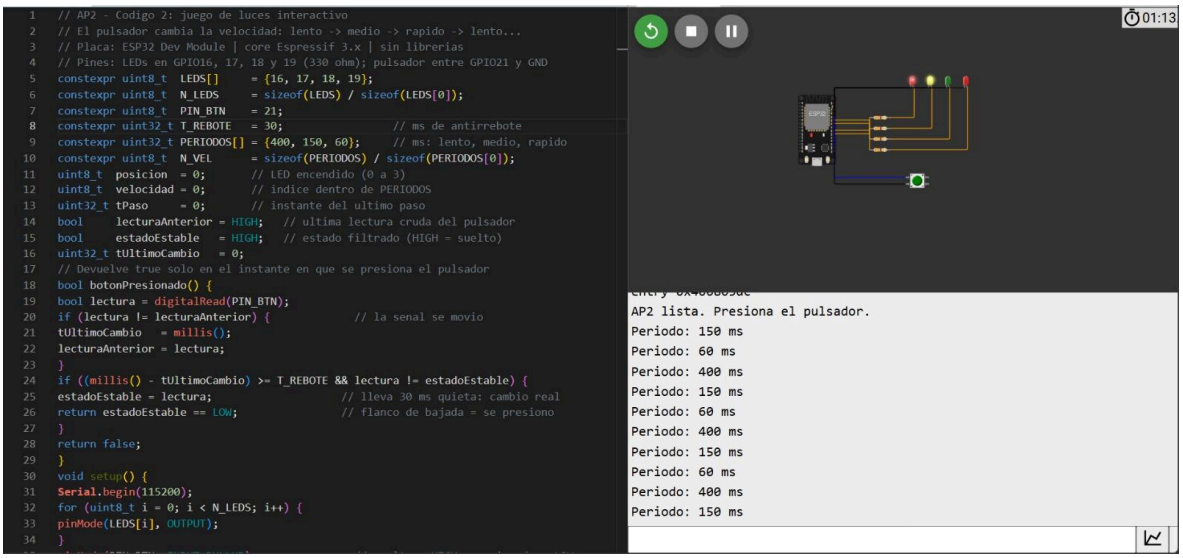


Tabla B. Efecto del antirrebote (10 clics)

T_REBOTE (ms)	Líneas impresas	¿Coincide con los 10 clics?
0	71	no
30	10	si

5.3 Parte C.

C1. Con el USB desconectado, armen el circuito en la protoboard según la Tabla de conexiones. Revisen la polaridad: pata larga del LED hacia la resistencia y el GPIO, pata corta hacia GND.

C2. Conecten el cable USB. En Arduino IDE elijan Herramientas → Placa → ESP32 Dev Module y el puerto que aparezca (COMx en Windows, /dev/ttyUSB0 en Linux).

C3. Abran el Código 2 y presionen Subir. Si aparece Connecting.... y no avanza, mantengan presionado el botón BOOT de la placa hasta que empiece a cargar.

C4. Abran el Monitor Serie a 115200 baudios y repitan los pasos B2 y B3. Resultado esperado: el mismo comportamiento que en Wokwi.

C5. Graben un video corto (30 s como máximo) del montaje funcionando.
https://canva.link/27dz7k17drkeu22

5.4 Parte D. Entrega

Enlace al Github
vicfer-nando/AP2-SistemasDigitales--Roa-Carrion--Maldonado-Machuca--Galvez-Merchan--Palma-Samaniego

6. Preguntas de control

1. ¿Por qué el Código 1 usa millis() en lugar de delay()? ¿Qué pasaría con el pulsador del Código 2 si cada paso de los LEDs usara delay(400)?

Usamos millis() porque realiza una temporización no bloqueante, lo que permite que el microcontrolador siga ejecutando otras instrucciones dentro del loop(). Si en el Código 2 pusiéramos un delay(400) en cada secuencia de los LEDs, el ESP32 quedaría completamente congelado durante esos 400 ms en cada paso. Si en ese lapso presionamos el pulsador, el sistema simplemente no detectaría la pulsación a menos que justo coincidiera con el ínfimo instante en el que termina el delay().

2. El pulsador va entre el GPIO21 y GND, sin resistencia externa. ¿Qué valor lee digitalRead() con el pulsador suelto y con el pulsador presionado? ¿Por qué?

- Suelto: Lee un nivel alto (HIGH o 1).
- Presionado: Lee un nivel bajo (LOW o 0).

Esto sucede porque configuramos el pin como INPUT_PULLUP. Esta función activa la resistencia interna del ESP32 conectada a 3.3V, lo que mantiene la línea en HIGH cuando el circuito está abierto. Al presionar el botón, cerramos la conexión a GND mandando el voltaje a 0V (LOW).

3. ¿Qué pasaría si conectaran el pulsador al GPIO34 sin cambiar el código? Justifiquen con lo visto en clase

El código fallaría o el pulsador no respondería. El GPIO34 pertenece al grupo de pines “Input Only” del ESP32 y no cuenta con resistencias internas de pull-down. Como el código usa INPUT_PULLUP, el pin se quedaría flotando internamente, generando lecturas erráticas o lectura constante en LOW, por lo que se necesitaría obligatoriamente una resistencia física externa en la protoboard para funcionar.

4. En el paso B3 la velocidad cambió una sola vez aunque mantuvieron presionado el pulsador. ¿Qué parte del Código 2 lo garantiza?

```
if ((millis() - tUltimoCambio) >= T_REBOTE && lectura != estadoEstable) {
estadoEstable = lectura;                    // lleva 30 ms quieta: cambio real
return estadoEstable == LOW;                // flanco de bajada = se presiono
}
```

Cuando el pulsador se mantiene presionado, estadoEstable ya es LOW, por lo que la condición lectura != estadoEstable deja de cumplirse. Así, la función devuelve true únicamente una vez por cada pulsación, evitando que la velocidad cambie repetidamente mientras se mantiene presionado el botón.

5. Expliquen la Tabla B: ¿por qué con T_REBOTE = 0 puede haber más líneas impresas que clics?

Con T_REBOTE = 0, se desactiva prácticamente el tiempo de antirrebote. Los contactos mecánicos del pulsador pueden generar varias transiciones rápidas entre HIGH y LOW en una sola pulsación.

6. Calculen la corriente que circula por un LED rojo (Vf ≈ 2,0 V) con una resistencia de 330 Ω a 3,3 V. ¿Por qué no conviene usar un LED azul?

Calculamos la corriente usando la ley de Ohm:

$$I = \frac{V_{fuente} - V_f}{R}$$

Sustituimos los valores:

$$I = \frac{3{,}3V - 2{,}0V}{330\Omega} = \frac{1{,}3}{330}$$

$$I = 3{,}94\ mA$$

Un LED azul suele tener una caída de tensión directa mayor, aproximadamente de 3,0 a 3,3 V. Como la alimentación es de 3,3 V, quedaría muy poca tensión para la resistencia, por lo que circularía una corriente muy pequeña o el LED podría no encender de forma adecuada.

7. Mencionen una diferencia entre probar el circuito en Wokwi y probarlo en la placa real.

En Wokwi los componentes operan bajo un modelo idealizado por software donde no hay ruido electromagnético, pérdidas por falso contacto en los cables dupont o variaciones de voltaje en la fuente. En la placa real entran en juego

factores físicos como la tolerancia real de las resistencias, el comportamiento térmico del ESP32 y la calidad del contacto físico del pulsador en la protoboard.

8. El ESP32 sigue funcionando a 240 MHz aunque el juego solo cambie de LED cada 400 ms. Propongan una idea para reducir el consumo de energía si el juego funcionara con batería.

Una solución práctica es reducir la frecuencia de reloj del procesador (por ejemplo, bajarla a 80 MHz o menos mediante setCpuFrequencyMhz()), ya que el control de temporizaciones de LEDs no requiere mucha capacidad de cómputo. Otra alternativa es configurar temporizadores por interrupciones (Hardware Timers) o utilizar el modo de bajo consumo Light Sleep entre cada paso de luz, despertando el ESP32 únicamente cuando el temporizador cumpla el período o cuando el pulsador genere una interrupción externa en el pin GPIO21.

7. Conclusiones

- La implementación de la temporización no bloqueante mediante la función millis() resulta indispensable en sistemas embebidos donde se requiere multitarea, ya que a diferencia de delay(), permite mantener liberado el ciclo principal del procesador para atender eventos como la lectura del pulsador en tiempo real.

- La técnica de antirrebote (debounce) por software combinada con la detección de flancos es esencial para garantizar la precisión en la lectura de entradas digitales mecánicas. En las pruebas experimentales se comprobó que sin este filtro (T_REBOTE = 0), un total de 10 clics mecánicos produjo 71 lecturas falsas en la consola, mientras que al aplicar un tiempo de estabilización de 30 ms se obtuvo exactamente una lectura por cada pulsación.

- El correcto aprovechamiento de las características internas de los pines GPIO del ESP32, como las resistencias de pull-up configuradas mediante software, reduce la necesidad de componentes externos en la protoboard; sin embargo, requiere tomar en cuenta las restricciones físicas de la placa, evitando usar pines sin pull-up interna (como el GPIO34) o pines con funciones especiales de arranque (strapping pins).

8. Declaración de Uso de Inteligencia Artificial

Se utilizó la herramienta de inteligencia artificial generativa ChatGPT para contar las líneas de texto que contenían la palabra «Periodo» en los datos recopilados. Su uso se limitó a esta tarea de conteo, sin intervenir en la elaboración ni modificación del contenido del trabajo.

9. Bibliografía / Referencias

[1] Internet Society, “In Memoriam: Jon Postel,” isoc.org, s. f. [En línea]. Disponible: https://www.internetsociety.org/. [Accedido: 23-oct-2026].

[2] V. Cerf, “I Remember IANA,” RFC 2468, IETF, oct. 1998. [En línea]. Disponible: https://www.rfc-editor.org/rfc/rfc2468

10. Anexos

- **Link Woki**

https://notebook.google.com/notebook/93c7b0ab-2a6b-4053-8ce2-e9dc6e509f0c?authuser=1&pageId=none